

HALAH

History Access Link for Authorised Healthcare

Implementation, Deployment and Evaluation on Ethereum Sepolia (Version 1.2)

Authors: Charles, Yasir, Daniel, Kejia, Yasmin, Farookh

Evaluation run: 24 September 2026

Table 1. *Summary of the evaluated deployment.*

Item	Value
Network	Ethereum Sepolia testnet (chainId 11155111)
Smart contract	ConsentSBTv3 at 0x0EC3617581DEC63A7f2bb162DC14d088916ADc73
Contract owner (issuer)	0xD2f6d7A4...b03552DCF, project test wallet (MetaMask)
Compilation	Remix, solc 0.8.34 (source pragma ^0.8.20), optimizer 200 runs, OpenZeppelin 4.9.6
Application	Python 3.11, Streamlit, web3.py, SQLite audit ledger; run in GitHub Codespaces
Zero-knowledge component	Groth16 over BN254; circom compiler 2.1.9 (circuit pragma 2.1.6), snarkjs 0.7.4
Code version	Repository branch <code>claude/pensive-tesla-u9uk4j</code>
Evidence	<code>evidence/sepolia_deploy_v3_2026-09-24.txt</code> , <code>evidence/sepolia_audit_ledger_2026-09-24.csv</code>
Time base	Ledger timestamps are server time of the Codespace (UTC, consistent with the Etherscan times); shown as hh:mm:ss

Introduction: HALAH System and Workflow

The HALAH implementation provides a role-based patient consent management environment that operationalises the proposed multi-party healthcare data-sharing framework. It is organised around four actors: the **patient**, who creates consent; the **guardians**, who approve it cryptographically; the **doctor**, who requests access to electronic medical record (EMR) modules; and the **auditor**, who reconstructs the consent lifecycle. Consent is represented not as a single approval event but as a lifecycle: it is created, independently authorised, anchored on a public blockchain, evaluated at each access request, used to control access, and finally revoked.

Version 1.2 differs from the previously reported prototype in three respects. First, activation and revocation are **transactions on the Ethereum Sepolia testnet**, so these two state changes can be checked by any third party on a public block explorer. Second, the access decision evaluates every component of the consent tuple, with revocation (E) assessed within the authorisation check VA. Third, defects found during end-to-end testing were corrected, and the principal corrections were re-validated on the live network (Section 11).

The consent object follows the representation used throughout the HALAH framework:

$$C = (I, A, P, T, L, E)$$

where **I** denotes the identities involved (patient, requester, guardians), **A** the authorisation evidence (guardian signatures and the on-chain token), **P** the purpose and data scope, **T** the temporal validity window, **L** the jurisdiction, and **E** the enforcement state (active or revoked). The evaluated workflow is:

Create → *Guardian Approval* (*N* – *of* – *N*) → *On-chain Anchor* (*SBT*)
→ *Access Decision* → *Revocation* → *Audit* → *Independent Verification*

The chapter follows this sequence. Each step is documented with a screenshot of the running system, the corresponding audit-ledger record and, where the step produces a blockchain transaction, its hash on Sepolia. Section 10 then evaluates the implementation systematically, and Section 14 discusses its limitations.

1. HALAH System and Deployment Environment

The HALAH application presents the system as a Patient Consent Management Platform. The blockchain and cryptographic mechanisms are embedded in the user workflow rather than exposed as separate tools: the application performs identity checks, signature verification, consent-state evaluation, on-chain anchoring and audit recording, while each participant sees only the actions relevant to their role.

1.1 Smart-contract deployment

The consent token contract **ConsentSBTv3** was compiled in Remix and deployed to Sepolia from the project test wallet through MetaMask (Figure 1). It implements a soulbound ERC-721 token: transfers between accounts are blocked in `_beforeTokenTransfer`, and burning follows the ERC-5484 authorisation interface. Minting is restricted to the owner and rejects zero addresses, a start time later than the expiry, and an expiry in the past. Each token stores the pseudonymous patient and requester addresses, the keccak256 hash of the purpose, the validity window `notBefore/expiry`, a two-letter jurisdiction code and a revocation flag. The T, L and E components of the consent are therefore recorded on-chain and can be evaluated by any party through the view functions `checkValid` (E and T) and `checkValidAt` (additionally L).

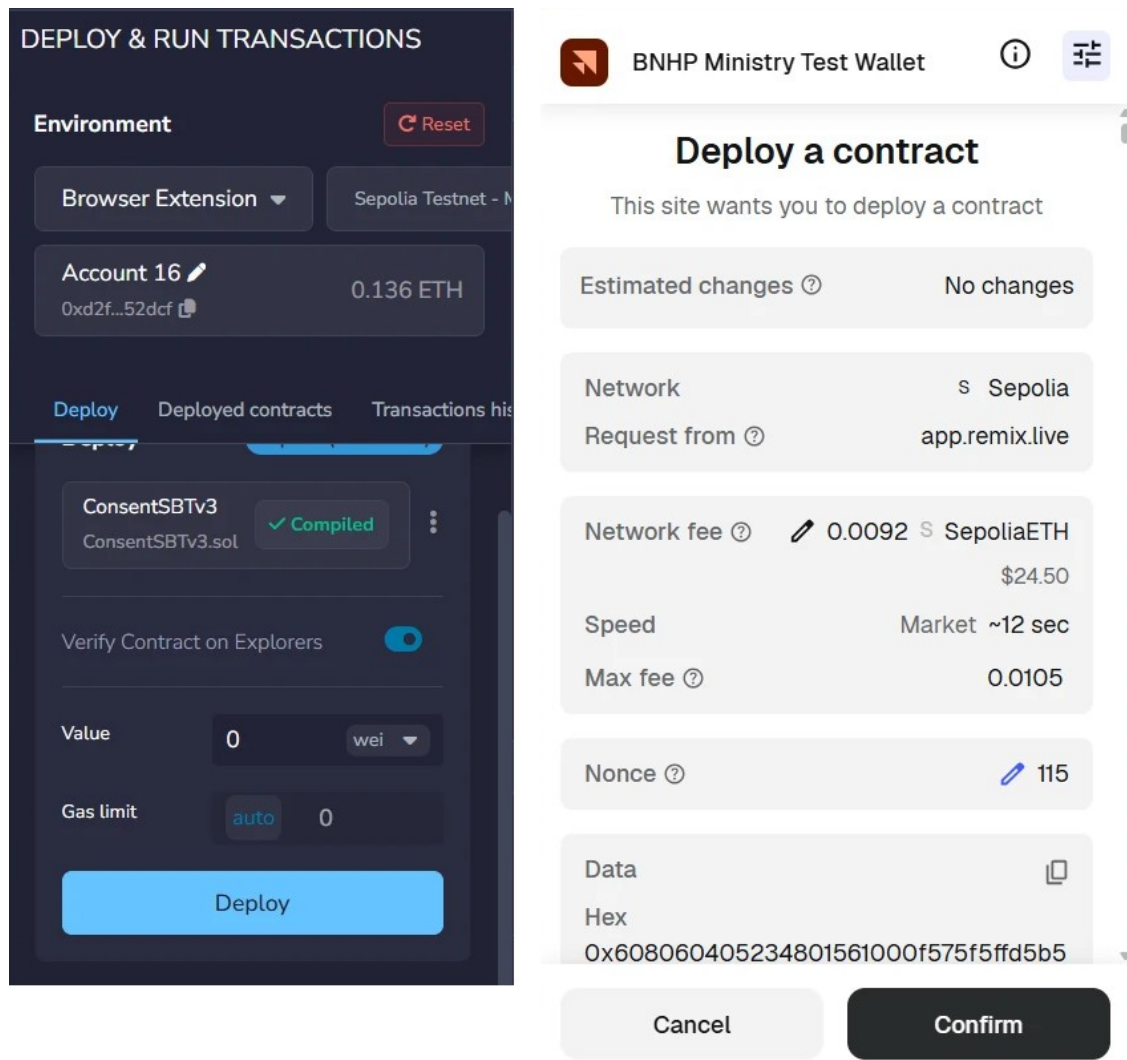


Figure 1. Deployment of ConsentSBTV3: the Remix Deploy panel (Sepolia, compiled contract) and the MetaMask confirmation (estimated network fee 0.0092 SepoliaETH, nonce 115).

After confirmation the contract is listed on Sepolia at [0x0EC3617581DEC63A7f2bb162DC14d088916ADc73](https://sepolia.etherscan.io/address/0x0EC3617581DEC63A7f2bb162DC14d088916ADc73). Figure 2 shows the creator wallet, the token tracker **MedicalConsent (CONSENT)** and the first application transaction. According to the Remix deployment log, the source was also verified on Blockscout.

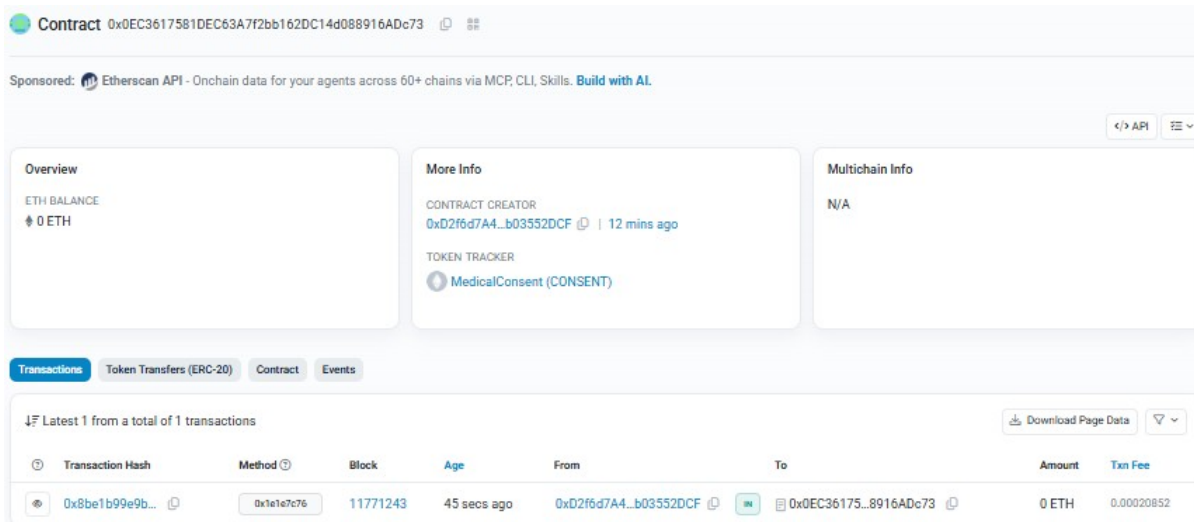


Figure 2. The deployed contract on Sepolia Etherscan: creator wallet, MedicalConsent (CONSENT) token tracker and the first mint transaction in block 11,771,243 (fee 0.00020852 ETH).

1.2 Application environment

To provide a reproducible, pre-configured execution environment, the application was run in GitHub Codespaces. The container configuration (.devcontainer/devcontainer.json) sets the RPC endpoint, the contract address and the contract version, and declares the owner key as an encrypted Codespaces secret, so no secret material is stored in the repository. Before the run, the readiness check `python -m blockchain.sepolia_check` was executed. It passes only if the chain is Sepolia (chainId 11155111), the signer has a positive balance, the address contains contract code and the signer is the contract owner(); it reported **READY** (Figure 3).

```
READY - run the app and complete a 2/2 guard
signature to mint on-chain.
@yasir12345678900 → /workspaces/HealthRWA-S
Personal-try (claude/pensive-tesla-u9uk4j)
```

Figure 3. Readiness check in GitHub Codespaces on the evaluated branch: READY for the new contract.

1.3 Reproducibility

The evaluation can be repeated from the repository with the following commands (the owner key is supplied as the PRIVATE_KEY secret or environment variable):

```
git checkout claude/pensive-tesla-u9uk4j && pip install -r requirements.txt
bash zk/build_zk.sh # builds the Groth16 circuit and keys
python -m blockchain.sepolia_check # network, balance, code, owner
python -m pytest -q tests/ # 23 automated tests
streamlit run app.py # the four role subsystems
python -m tools.verify_poc # ledger versus Sepolia
```

2. Create Smart Consent Contract: Patient Subsystem (SRQ1)

The first functional subsystem represents the patient-side initiation of a consent transaction and corresponds directly to the construction of the consent object. The patient specifies the fields in Table 2, each of which operationalises one component of C.

Table 2. Mapping of patient input fields to the consent tuple.

Input field	Component	Role in the consent
Patient DID	I	Identity pointer of the data subject (did:patient method, synthetic Synthea identifier)
Authorised Doctor DID	I	Binds the authorisation to one requester (did:hospital:001)
Data Access Scope	P	Restricts access to selected EMR modules; the purpose is fixed to Treatment in this prototype
Designated Guardian DIDs	A	Parties whose signatures are required; threshold = number of guardians
Start / Expiry time	T	Validity window, later stored on-chain as notBefore / expiry
Jurisdiction	L	ISO-3166 code stored with the consent and on-chain (AU in the evaluated consent)

Version 1.2 validates the input before the consent is stored: a consent with an empty data scope is rejected, and each guardian entry must be a DID. The evaluated consent f050c29a-3d92-4131-8605-63bcb1ff8d92 was created at 09:50:49 for the modules Observation, Medication, Condition and Procedure, with two guardians and jurisdiction AU (Figure 4). At this point the consent exists only in the application ledger, in the state **PENDING_SIGNATURE**; nothing is written to the blockchain until the guardians have approved.

Figure 4. Patient subsystem: a consent created with four EMR modules, two guardians and jurisdiction AU, in the state `PENDING_SIGNATURE` (reproduction lifecycle, consent 04da25f5).

Because a revoked consent cannot be photographed in its earlier states, Figure 4, Figure 5 and Figure 8 were captured from a second, identical lifecycle executed after the evaluation (consent 04da25f5 for patient `did:patient:5f01f823-...`, anchored as token #2 in block 11,771,581, gas 179,306). All ledger rows and transactions cited in the text remain those of f050c29a.

This module therefore corresponds to **Consent Object Construction and Patient Initiation**.

3. Guardian Cryptographic Approval (SRQ2)

The guardian subsystem operationalises the multi-party authorisation component. A guardian enters the target Consent ID, the system retrieves the pending consent, and the guardian approves it with an **EIP-712 typed-data signature** over the following structure:

$$domain = (\text{HALAH Consent}, 1, \text{chainId } 11155111)$$

$$Approval = \{ \text{consentId} : \text{string}, \text{guardian} : \text{string} (DID), \text{action} : \text{APPROVE} \}$$

The signature binds the approval to the consent, the guardian DID and the network; because the domain has no `verifyingContract` field it is not bound to a specific contract. The application recovers the signer off-chain (`web3.py recover_message`) and compares it with the address associated with the guardian DID. In this prototype that address, and the signing key, are derived deterministically from the DID, so the application signs on the guardian's behalf (see Section 14). Signatures for another consent, another guardian or another chain fail verification, and the interface refuses a second signature from the same guardian. Activation requires every designated guardian:

$$\sum \{ g \in G \} \text{VerifySig}(g, C) \geq \text{threshold} = 2 \vee (here \vee G \vee 2)$$

In the evaluated run the two approvals were recorded at 09:51:05 and 09:51:07 from two distinct signer addresses (0x3Af549...c570 for guardian A and 0x39B12b...C90d for guardian B). When the threshold is reached the application records a provisional local identifier (MINT_SBT, SBT-LOCAL - f050c29a) and starts a background job that calls mintConsentV3 (selector 0x1e1e7c76) from the owner account. The transaction was confirmed in block **11,771,243** at 09:51:12, used 196,406 gas and produced **token #1** (displayed in the application as SBT-1), which replaced the provisional identifier. The token is held by the pseudonymous patient address 0x789997BF...E30C5, the last 20 bytes of keccak256 of the patient DID (Figure 5, Figure 6).

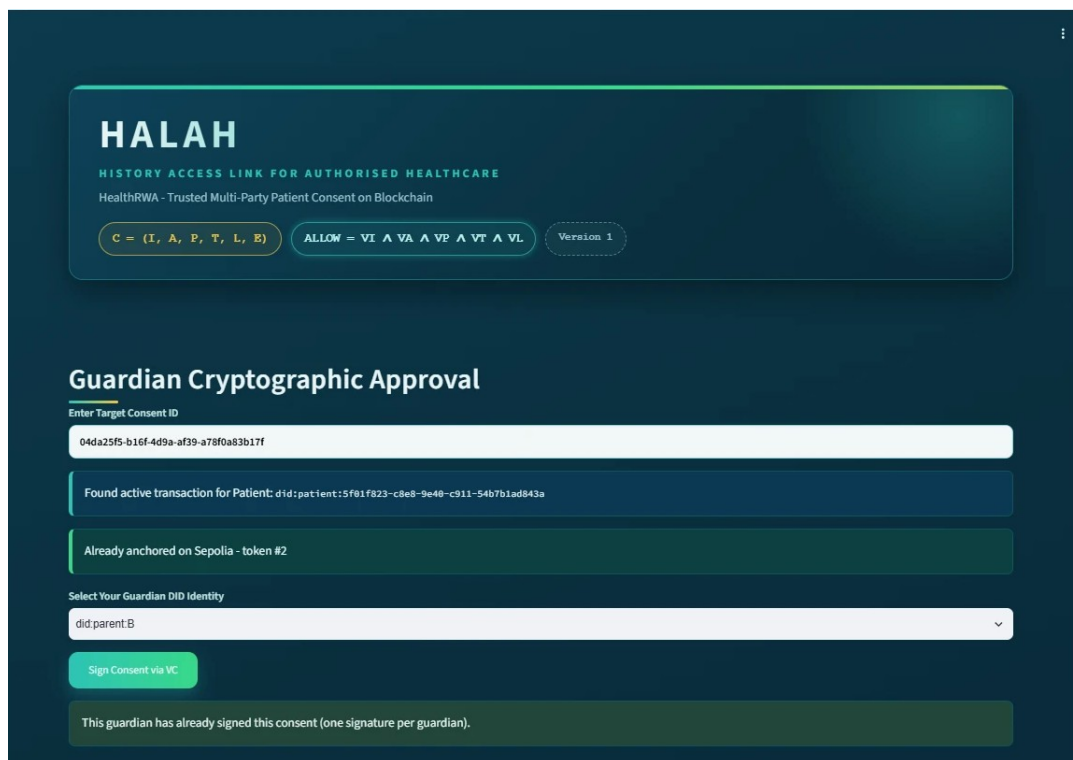
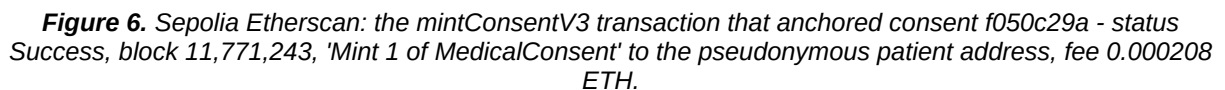


Figure 5. Guardian subsystem after the 2/2 threshold: 'Already anchored on Sepolia - token #2' (reproduction lifecycle; f050c29a was anchored identically as token #1).

The mint transaction can be inspected at [Etherscan: mint transaction](#).



4. Secure Patient Data Access for Doctor (SRQ3–SRQ5)

$$ALLOW = VI \wedge VA \wedge VP \wedge VT \wedge VL$$

Check	Condition evaluated in version 1.2
VI – identity	Patient and requester identifiers are DIDs, at least one signature exists and every recorded guardian signature verifies (EIP-712)
VA – authorisation	Signatures $\geq$ threshold, a token identifier is present and the consent is not revoked (local state)
VP – purpose	The consent purpose is in the permitted set {Treatment, Research, Emergency}
VT – time	Start $\leq$ now $\leq$ expiry, evaluated on the consent's stored dates
VL – jurisdiction	The requester's declared jurisdiction equals the consent's jurisdiction (an empty consent

Check	Condition evaluated in version 1.2
	jurisdiction is unrestricted)

The screenshot shows a web interface for the Doctor subsystem. At the top, there's a field 'Enter Authorised Consent ID' with the value 'f050c29a-3d92-4131-8605-63bcb1ff8d92'. Below this is a section titled 'Consent Information' with three cards: 'Patient' with 'did:pat...', 'Consent ID' with 'f050c2...', and 'Purpose' with 'Treat...'. Underneath is a dropdown for 'Requester jurisdiction (VL check)' set to 'AU'. A 'Decision matrix' section shows a table where all five checks (VI, VA, VP, VT, VL) are 'PASS', leading to an 'ALLOW' result. At the bottom, a green bar indicates 'Consent State: ACTIVE'.

Decision matrix - ALLOW = VI AND VA AND VP AND VT AND VL					
VI	VA	VP	VT	VL	ALLOW
PASS	PASS	PASS	PASS	PASS	PASS

Consent State: ACTIVE

Figure 7. Doctor subsystem, requester jurisdiction AU: all five checks PASS, ALLOW = PASS and the consent state is ACTIVE.

The doctor then selects the modules to request. The request must be non-empty and a subset of the authorised scope, and the matrix must return ALLOW. The request also generates and verifies a **Groth16 proof** for the circuit below, in which commitment and now are public and the secret s and the expiry e are private witnesses:

$$\exists(s, e): \text{Poseidon}(s, e) = \text{commitment} \wedge \text{now} \leq e$$

In the evaluated run the doctor requested all four modules twice, at 09:51:38 and 09:51:41, and both requests were **GRANTED** with `zk_proof_verified = true` and `proof_mode = groth16`. This confirms that the Groth16 proof was generated and verified, rather than the hash-based fallback used when the circuit artefacts are unavailable (Figure 8). Its contribution should be interpreted carefully: the prover and verifier run on the same server, the commitment is not anchored on-chain, and the expiry is already public in the token. The proof therefore demonstrates that the ZK toolchain is integrated end-to-end, but it does not yet add a privacy or trust guarantee beyond VT (Section 14).

Enter Authorised Consent ID

f050c29a-3d92-4131-8605-63bcb1ff8d92

Consent Information

Patient
did:pat...

Consent ID
f050c2...

Purpose
Treat...

Requester jurisdiction (VL check)

SA

Decision matrix - ALLOW = VI AND VA AND VP AND VT AND VL

VI	VA	VP	VT	VL	ALLOW
PASS	PASS	PASS	PASS	FAIL	FAIL

Consent State: ACTIVE

Figure 9. Requester jurisdiction SA against a consent issued for AU: VL = FAIL and ALLOW = FAIL while the consent state remains ACTIVE.

On submission, access is refused and the failed check is named (Figure 10). The ledger entry at 10:02:51 records **DENIED** together with the complete decision matrix, so the refusal is itself auditable. In version 1.1 this request would have been granted, because VL was displayed but not included in the decision.

Request Patient Data

Select Request Scope

Observation × Medication × Condition × Procedure ×

Requested Modules: Observation, Medication, Condition, Procedure

Request Data Access (Groth16 ZK proof)

Security Exception: Access Denied by ZK-Policy. Failed checks: VL

Figure 10. Access denied: 'Security Exception: Access Denied by ZK-Policy. Failed checks: VL'.

This test therefore corresponds to **Jurisdictional Constraint (L)** and **Fail-Closed Access Control**; because the jurisdiction is self-declared, it evidences the policy logic rather than protection against a dishonest requester.

6. Consent Revocation

The enforcement component E requires that a patient can withdraw consent. In the patient's **My Consents** view each consent is listed with its state, signature count and token. Pressing **Revoke** on consent f050c29a first updates the application state and then submits revoke(1) (selector 0x20c5429b) from the owner account (Figure 11); the patient's own address has no key, so revocation is relayed by the issuer.



Figure 11. The patient revokes consent f050c29a (token #1); the application waits for the Sepolia block confirmation. The other rows are earlier trial consents.

The revocation was confirmed in block **11,771,307** at 10:04:00 and the consent is shown as REVOKED (Figure 12). From then on, the doctor subsystem reports the consent as not active because VA fails (Figure 13), and the on-chain revocation flag of token #1 was later read back as true by the independent verification (Section 9). Two properties should be noted: the application's own decision uses its local state, which is written first, and revocation cannot recall data already released to the requester.

My Consents

2502632b... → did:hospital:001	REVOKED	0/2 sigs - token: -	REVOKED
aca837a1... → did:hospital:001	REVOKED	0/2 sigs - token: -	REVOKED
cd9a24bd... → did:hospital:001	REVOKED	2/2 sigs - token: SBT-LOCAL- cd9a24bd	REVOKED
011bbb21... → did:hospital:001	REVOKED	2/2 sigs - token: SBT-8	REVOKED
0975c645... → did:hospital:001	PENDING_SIGNA TURE	0/2 sigs - token: -	<button>Revoke</button>
f050c29a... → did:hospital:001	REVOKED	2/2 sigs - token: SBT-1	REVOKED

Figure 12. After confirmation, consent f050c29a is REVOKED with token SBT-1 (token #1).

Secure Patient Data Access

Enter Authorised Consent ID

f050c29a-3d92-4131-8605-63bcb1ff8d92

Consent Information

Patient did:patient:fe62...	Consent ID f050c29a-3d92-...	Purpose Treatment
---------------------------------------	--	-----------------------------

Requester jurisdiction (VL check)

AU

Decision matrix - ALLOW = VI AND VA AND VP AND VT AND VL

VI	VA	VP	VT	VL	ALLOW
PASS	FAIL	PASS	PASS	PASS	FAIL

Consent is not active. Current state: REVOKED

The Doctor cannot access patient data because the consent is not currently valid.

Figure 13. Doctor subsystem after revocation (negative test N2): VA = FAIL, ALLOW = FAIL and 'Consent is not active. Current state: REVOKED'.

The revocation transaction is available at [Etherscan: revoke transaction](#).

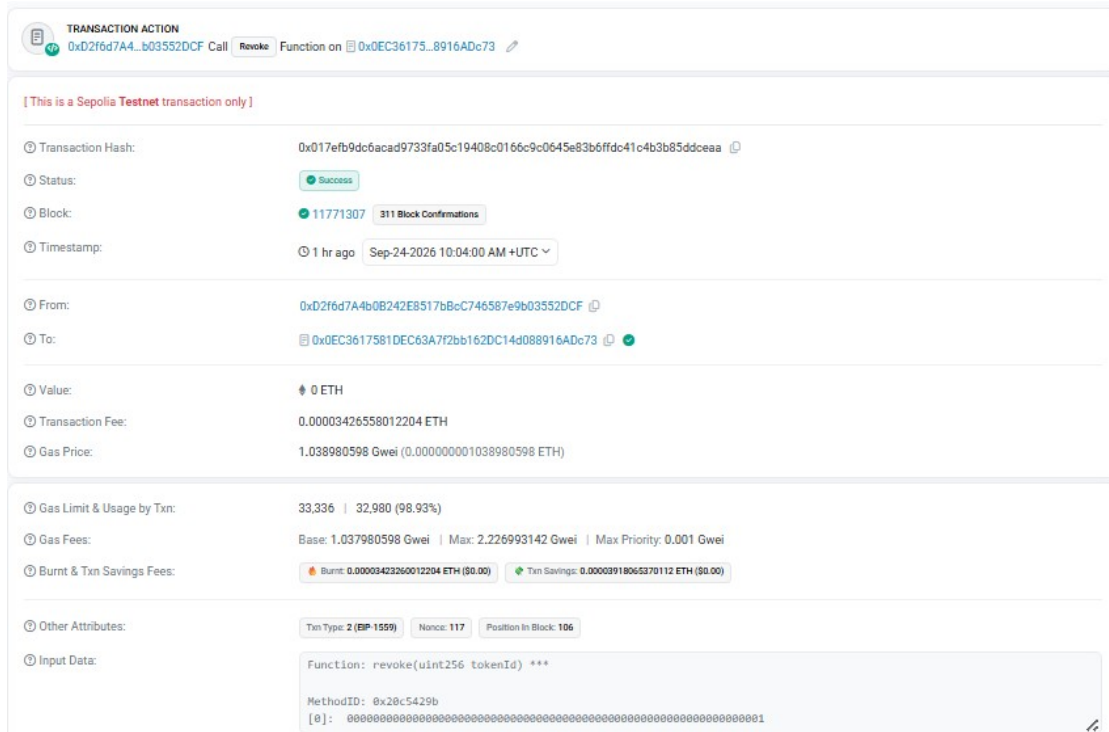


Figure 14. Sepolia Etherscan: the revoke transaction for token #1 - status Success, block 11,771,307, decoded input 'Function: revoke(uint256 tokenId)' with tokenId = 1 (selector 0x20c5429b).

The revocation subsystem therefore corresponds to **Enforcement State Management (E)** and **Consent Withdrawal**.

7. Blockchain Audit & Consent Lifecycle for Auditor (SRQ6)

The auditor subsystem provides an independent view of the consent process. It shows aggregate indicators (total events, consent events, access requests and on-chain anchors) and, for a selected consent, reconstructs the chronological lifecycle (Figure 15).

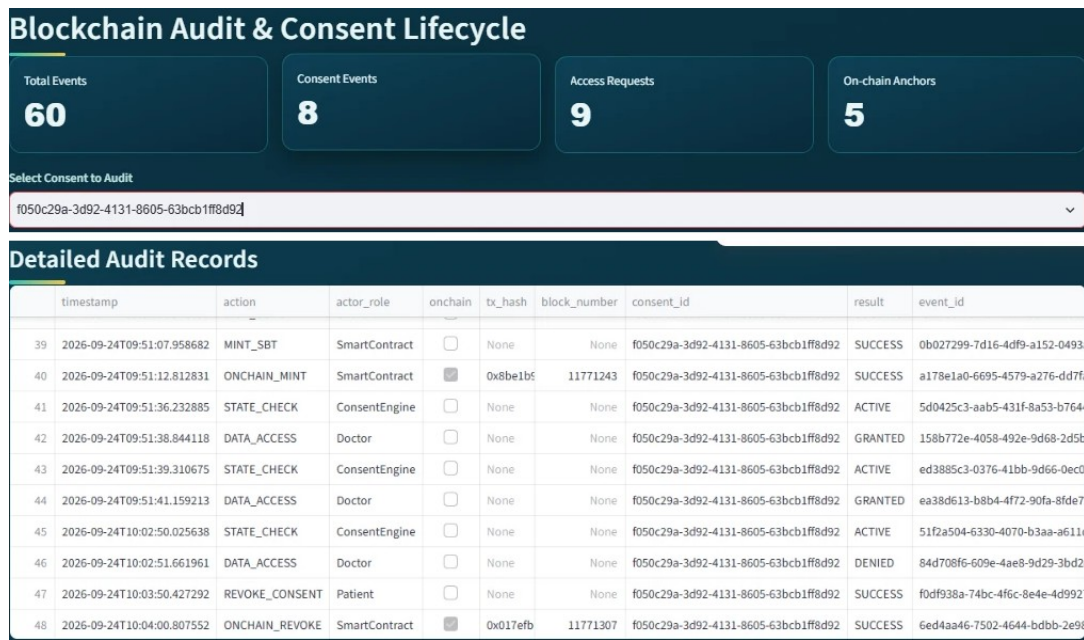


Figure 15. Auditor subsystem with consent *f050c29a* selected: aggregate indicators (60 events, 5 on-chain anchors) and the Detailed Audit Records, in which *ONCHAIN_MINT* and *ONCHAIN_REVOKE* carry their transaction hashes and blocks.

Table 4 lists every ledger event for consent *f050c29a* exactly as exported from the running system. Events backed by a blockchain transaction carry the transaction hash and block; the other events carry a SHA-256 event digest in the ledger. The *MINT_SBT* row records the provisional identifier assigned at the 2/2 threshold, superseded by token #1 when the *ONCHAIN_MINT* receipt is decoded five seconds later.

Table 4. Audit ledger of consent *f050c29a*, 24 September 2026.

Time	Action	Role	Actor	Result	Evidence
09:50:49	CREATE_CONSENT	Patient	did:patient:fe62...0d82	SUCCESS	-
09:51:05	SIGN_CONSENT	Guardian	did:parent:A	SUCCESS	EIP-712 signer 0x3Af549...c570 · 1/2
09:51:07	SIGN_CONSENT	Guardian	did:parent:B	SUCCESS	EIP-712 signer 0x39B12b...C90d · 2/2
09:51:07	MINT_SBT	SmartContract	Blockchain	SUCCESS	local id SBT-LOCAL-f050c29a
09:51:12	ONCHAIN_MINT	SmartContract	Blockchain	SUCCESS	tx 0x8be1b99e...daf13d · block 11771243
09:51:36	STATE_CHECK	ConsentEngine	System	ACTIVE	VI=T VA=T VP=T VT=T VL=T ALLOW=T
09:51:38	DATA_ACCESS	Doctor	did:hospital:001	GRANTED	proof=groth16, zk=True
09:51:39	STATE_CHECK	ConsentEngine	System	ACTIVE	VI=T VA=T VP=T VT=T VL=T ALLOW=T
09:51:41	DATA_ACCESS	Doctor	did:hospital:001	GRANTED	proof=groth16, zk=True
10:02:50	STATE_CHECK	ConsentEngine	System	ACTIVE	VI=T VA=T VP=T VT=T VL=F ALLOW=F
10:02:51	DATA_ACCESS	Doctor	did:hospital:001	DENIED	zk=True, VL=F
10:03:50	REVOKE_CONSENT	Patient	did:patient:fe62...0d82	SUCCESS	-
10:04:00	ONCHAIN_REVOKE	SmartContract	Blockchain	SUCCESS	tx 0x017efb9d...ddceaa · block 11771307

The ledger yields an explicit chain of evidence:

$$Patient \rightarrow Guardian_A \rightarrow Guardian_B \rightarrow SmartContract(mint) \rightarrow ConsentEngine$$

$$\rightarrow Doctor(GRANTED) \rightarrow Doctor(DENIED, VL) \rightarrow Patient(revoke) \rightarrow SmartContract(revoke)$$

An auditor can therefore relate each access decision to the approvals, the on-chain activation and the decision matrix in force at that moment, and so establish why access was permitted or refused. Because the requester is not authenticated in this prototype, the identity recorded for an access event is the consent's requester DID rather than a proven identity.

The auditor subsystem therefore corresponds to **Lifecycle Traceability and Accountability**.

8. Detailed Audit Records and Complete Ledger

The detailed audit view exposes the event-level attributes (actor, role, action, result, timestamp, consent, transaction hash, block number and metadata), while the lifecycle view communicates the sequence. This reflects different stakeholder needs: a clinician needs to know whether a consent is valid, whereas an auditor must establish when an approval occurred, which identity performed it and which decision followed.

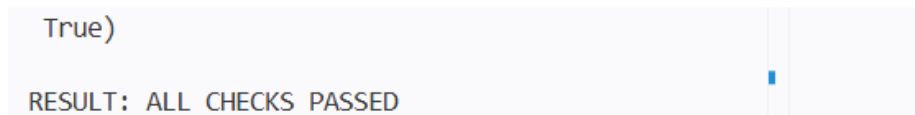
The complete ledger at the end of the run contained 49 events across seven consents created on 22 and 24 September, including four on-chain transactions: the mint and revocation of token #1 on the current contract, and the mint and revocation of token #8 during a 22 September trial on the previous (v2) contract. It records creations, approvals, state checks, refusals and revocations, not only successful access. Off-chain events are protected by per-event digests, which detect accidental corruption but not deliberate modification by the operator; only the on-chain events are independently tamper-evident.

This component therefore corresponds to **Forensic Audit Information and Evidence Retention**.

9. Independent Verification Against Sepolia

To test whether the application ledger is consistent with the blockchain, the tool `python -m tools.verify_poc` reads the ledger and, for every event that carries a transaction hash, fetches the receipt from Sepolia and checks that it exists, has status success and was included in the recorded block. For each consent anchored on the current contract it then reads the token and reports the owner, requester, purpose hash, validity window, jurisdiction and `checkValid/checkValidAt` results; the pass/fail verdict additionally requires that the local revocation state equals the on-chain revocation flag. Tokens minted on earlier deployments are reported and skipped.

Run in Codespaces after the lifecycle, the tool verified the ledger transactions against Sepolia (five after the reproduction lifecycle), found the on-chain revocation flags matching the local state for both tokens, and reported **ALL CHECKS PASSED** (Figure 16, Figure 17).



```
True)
RESULT: ALL CHECKS PASSED
```

Figure 16. Final lines of the independent verification: the local revocation state matches the chain; **RESULT: ALL CHECKS PASSED**.

```

@yasir12345678900 → /workspaces/HealthRWA-Sepolia-
Personal-try (claude/pensive-tesla-u9uk4j) $ pytho
n -m tools.verify_poc
local ledger state revoked=false (matches chain
: True)

RESULT: ALL CHECKS PASSED
@yasir12345678900 → /workspaces/HealthRWA-Sepolia-
Personal-try (claude/pensive-tesla-u9uk4j) $ █

```

Figure 17. Final verdict of a later `verify_poc` run covering both lifecycles (token #2 not revoked, matching the chain): **RESULT: ALL CHECKS PASSED.**

Table 5. Register of Sepolia transactions in the ledger.

Operation	Transaction	Block	Function
Anchor consent f050c29a (token #1)	0x8be1b99e9b7054d427...	11,771,243	mintConsentV3 (0x1e1e7c76)
Revoke consent f050c29a (token #1)	0x017efb9dc6acad9733...	11,771,307	revoke (0x20c5429b)
Trial 22 Sep, v2 contract (token #8)	0x5f9b2f14a3c061ef6a...	11,757,272	mintConsent (v2)
Trial 22 Sep, v2 contract (token #8)	0x2adf73d854071e9010...	11,757,344	revoke (v2)
Reproduction lifecycle (token #2)	0x25ef5e2f0046735dcb...	11,771,581	mintConsentV3 (0x1e1e7c76)

This step therefore corresponds to **Ledger Integrity and Independent Verifiability**.

10. Evaluation

10.1 Test cases

Table 6 summarises the test cases. Live tests were executed on Sepolia through the user interface; automated tests are part of the repository and run with `python -m pytest -q tests/` (23 of 23 pass, including a Groth16 round-trip and tamper test).

Table 6. Test cases, expected and observed results.

ID	Scenario	Expected	Observed	Evidence
P1	Create consent (4 modules, 2 guardians, AU)	Stored, PENDING_SIGNATURE	As expected	Ledger 09:50:49; Figure 4
P2	Two guardian approvals	2 valid EIP-712 signatures	As expected	Ledger 09:51:05, 09:51:07
P3	Anchor on Sepolia	Token minted, id recorded	Token #1, block 11,771,243	Figure 2; Table 5
P4	Doctor request, AU	GRANTED with Groth16 proof	GRANTED ×2, proof_mode groth16	Figure 7; ledger 09:51:38
P5	Revoke	On-chain revoke, state REVOKED	Block 11,771,307	Figure 12; Table 5
P6	Independent verification	All transactions and revoke flag match	ALL CHECKS PASSED	Figure 16

ID	Scenario	Expected	Observed	Evidence
N1	Doctor request, SA (consent AU)	DENIED, VL fails	DENIED, 'Failed checks: VL'	Figure 9, Figure 10
N2	Doctor after revocation	Not active, VA fails	As expected	Figure 13
N3	Empty request scope	Refused	Refused (automated run)	Regression check
A1	EIP-712 wrong guardian / consent / missing signature	Rejected	Rejected	test_eip712_roundtrip_and_rejections
A2	Each of VI, VA, VP, VT, VL false in turn	ALLOW false	ALLOW false	test_matrix_..._fails_independently
A3	Tampered or expired ZK proof	Rejected / not provable	As expected	test_real_groth16_..., test_expired_...
A4	Revoked consent state, persistence across reload	REVOKED, persisted	As expected	test_phase_red.py (4 tests)
A5	Regression tests for the v1.2 corrections	Pass	5 of 5 pass	tests/test_regressions.py

10.2 Indicative measurements

Table 7. Indicative measurements from the single evaluated lifecycle ($n = 1$, Sepolia testnet).

Measure	Value	Source
Gas used, mintConsentV3	196,406 (token #1); 179,306 (token #2)	Ledger / receipts
Transaction fee, mint	0.00020852 ETH	Figure 2
Estimated deployment fee	≈ 0.0092 SepoliaETH (max 0.0105)	Figure 1
Threshold reached → mint confirmed	≈ 5 s (09:51:07 → 09:51:12)	Ledger
Revoke requested → confirmed	≈ 10 s (10:03:50 → 10:04:00)	Ledger
Ledger events per complete lifecycle	13	Table 4

These values are indicative only: they come from a single run on a public testnet whose block times and fees vary, and they are not a performance evaluation.

11. Implementation Defects Identified and Corrected

End-to-end testing of version 1.1 revealed defects that were not visible in unit tests because each component worked in isolation. The table states, for each correction, how it is evidenced.

Table 8. Defects found in version 1.1 and their correction in version 1.2.

Defect in v1.1	Consequence	Correction and evidence
Receipt decoding for v3 mints referenced an undefined name	On the v3 contract the token id was never recorded, so revocation was not sent on-chain and re-anchoring could mint duplicates (v2 mints were unaffected)	Import fixed; regression test; on Sepolia the ledger records token_id = 1 for f050c29a
Jurisdiction (VL) displayed but not part of ALLOW	Wrong-jurisdiction requests were granted	ALLOW required; regression test; DENIED on Sepolia (Section 5)
Chain reconciliation read notBefore as expiry	Existing v3 mints could not be matched	Correct field; regression test

Defect in v1.1	Consequence	Correction and evidence
v2-compatible mintConsent called itself externally	Always reverted with 'caller is not the owner'	Internal function; source check in tests; contract redeployed
Consent record written after _safeMint	A contract receiver could observe an empty record (not reachable with the keyless patient addresses used here)	Record written before minting
Empty request scope accepted	An access with no data was logged as GRANTED	Non-empty scope required
Three EMR source files partially corrupted	Observation, Medication and Procedure appeared empty	Valid rows recovered; complete re-export deferred (Section 14); regression test

12. Comparison with the Previous Version

Table 9. Capabilities of the previously reported version (1.1) and of version 1.2.

Aspect	Version 1.1	Version 1.2
Consent activation	Token displayed in the interface	Token minted on Sepolia; hash and block recorded
Revocation	Local state	Local state and on-chain revoke transaction
Jurisdiction (L)	Displayed	Enforced in ALLOW; stored on-chain
Temporal validity (T)	Local check	Local check; also stored on-chain (notBefore / expiry)
Guardian approval	Identity check message	EIP-712 signatures verified per guardian
Zero-knowledge component	Placeholder	Groth16 proof generated and verified (see limitations)
Negative testing	None reported	Jurisdiction refusal on Sepolia; automated negative tests
Independent verification	None	verify_poc against Sepolia: ALL CHECKS PASSED

13. Requirements Traceability and Framework Correspondence

Table 10. Traceability of the system requirements to the evidence in this chapter.

Req.	Requirement (role)	Subsystem	Evidence	Result
SRQ1	Patient creates scoped, time-bound consent	Create Smart Consent Contract	Section 2; P1; Figure 4	Met
SRQ2	Guardians approve with verifiable signatures	Guardian Cryptographic Approval	Section 3; P2–P3; Table 5	Met (prototype key model)
SRQ3	Access only while consent is valid	Secure Patient Data Access	P4, N1, N2; Figure 7–Figure 10	Met
SRQ4	Access limited to authorised scope	Secure Patient Data Access	Section 4; N3; scope subset check	Met
SRQ5	Cryptographic verification before release	Secure Patient Data Access	Groth16 proof_mode in ledger; A3	Partly (Section 14)
SRQ6	Auditable, verifiable consent lifecycle	Auditor; verify_poc	Sections 7–9; P6; Table 4	Met

Table 11. Framework components and their implementation.

HALAH framework component	Implementation	Evidence
Consent object construction (C)	Create Smart Consent Contract	Section 2, Table 2
Identity (VI)	DID checks and EIP-712 guardian signatures	Section 3
Multi-party consent (A)	N-of-N guardian threshold (2 of 2 evaluated)	Ledger 09:51:05 / 09:51:07
Consent activation	Soulbound token on Sepolia (ConsentSBTV3)	Block 11,771,243
Purpose and scope (VP)	Permitted purposes; non-empty scope subset	Section 4
Temporal validity (VT)	Local check; stored on-chain	Table 3
Jurisdiction (VL)	Enforced in ALLOW; stored on-chain	Section 5
Enforcement / revocation (E)	Local state and on-chain revoke()	Block 11,771,307
Traceability and accountability	Auditor lifecycle, detailed records, ledger	Sections 7–8
Ledger integrity	Independent verification of on-chain events	Section 9

14. Discussion and Limitations

The evaluation shows that the HALAH workflow operates end-to-end on a public network and that its principal decision rules, including a fail-closed refusal and an on-chain revocation, behave as specified. It also exposes the boundary between what the prototype demonstrates and what a deployment would require:

- **Trusted operator.** One application server holds the owner key, derives the guardian keys, generates and verifies the ZK proof and keeps the ledger. The blockchain provides public, timestamped evidence of activation and revocation, not decentralised enforcement.
- **Guardian keys.** The EIP-712 message format is final, but the expected signer is derived from the guardian DID with a public constant, so anyone who knows a guardian DID could produce a valid approval. Production use requires a DID-to-address registry (DID documents or verifiable credentials) and wallet-held keys.
- **Requester authentication.** The doctor role and jurisdiction are self-declared in the interface. A signed request or verifiable credential is required before deployment.
- **Zero-knowledge proof.** The Groth16 circuit is integrated and verified, but prover and verifier are the same server, the commitment is not anchored on-chain and the expiry is public. The proof therefore adds no privacy guarantee yet; anchoring the commitment and moving proof generation to the requester is future work.
- **On-chain metadata.** No EMR content is stored on-chain, but the purpose (visible in calldata), jurisdiction, validity window, event timing and stable patient/requester pseudonyms are public. Under GDPR Art. 4(5) and the Australian Privacy Act such pseudonymous data is likely still personal information; salted per-consent identifiers are recommended.
- **Decision source and revocation.** Access decisions use the application's local state rather than querying checkValid at access time; revocation is relayed by the issuer and cannot recall data already released.

- **Ledger integrity.** Off-chain events carry independent digests rather than a hash chain; periodic anchoring of a Merkle root of the ledger would make them tamper-evident.
- **Scope of the evaluation.** One lifecycle on a testnet with synthetic Synthea data (three files partially corrupted and pending re-export), a fixed purpose, and N-of-N guardians. The results demonstrate functional correctness, not performance or scalability.

15. Conclusion

Taken together, the subsystems demonstrate an end-to-end implementation of the HALAH authorisation workflow on Ethereum Sepolia. The patient defines the consent conditions; two guardians approve with verifiable signatures; the consent is anchored as a soulbound token; each access request is evaluated against every component of the consent tuple; an out-of-jurisdiction request is refused; the patient revokes the consent on-chain; and the recorded lifecycle is re-verified independently against the chain.

$C \rightarrow MS \rightarrow SBT_{chain} \rightarrow Decision(VI \wedge VA \wedge VP \wedge VT \wedge VL) \rightarrow EMR \rightarrow Revoke_{chain} \rightarrow Audit \rightarrow Verify$

From a research-implementation perspective, the main contribution of version 1.2 is the continuity between consent specification, multi-party authorisation, public activation and revocation, full-tuple enforcement and independently verifiable audit evidence. The limitations identified in Section 14 define the next steps: wallet-held guardian keys with a DID registry, authenticated requesters, an on-chain commitment for the zero-knowledge proof, chain-queried access decisions and a hash-chained ledger.

References

- Bloemen, R., Logvinov, L. and Evans, J. (2017) EIP-712: Typed structured data hashing and signing. Ethereum Improvement Proposals.
- Entriken, W., Shirley, D., Evans, J. and Sachs, N. (2018) ERC-721: Non-Fungible Token Standard. Ethereum Improvement Proposals.
- Cai, B. (2022) ERC-5484: Consensual Soulbound Tokens. Ethereum Improvement Proposals.
- Groth, J. (2016) 'On the size of pairing-based non-interactive arguments', EUROCRYPT 2016, LNCS 9666, pp. 305–326.
- Grassi, L. et al. (2021) 'Poseidon: A new hash function for zero-knowledge proof systems', USENIX Security 2021.
- Walonoski, J. et al. (2018) 'Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record', JAMIA, 25(3), pp. 230–238.
- W3C (2022) Decentralized Identifiers (DIDs) v1.0. W3C Recommendation.

Appendix A. Verification Links

Contract:

<https://sepolia.etherscan.io/address/0x0EC3617581DEC63A7f2bb162DC14d088916ADc73>

Mint of token #1:

<https://sepolia.etherscan.io/tx/0x8be1b99e9b7054d427bdee0a0b5cc56ae99030a399f4b11d3629ed5d98daf13d>

Revocation of token #1:

<https://sepolia.etherscan.io/tx/0x017efb9dc6acad9733fa05c19408c0166c9c0645e83b6ffdc41c4b3b85ddceaa>

Full ledger export: evidence/sepolia_audit_ledger_2026-09-24.csv; deployment and run record: evidence/sepolia_deploy_v3_2026-09-24.txt.